

Spécialité : Transformation Digitale Industrielle
Module : Génie Logiciel

COMPTE RENDU

TP 04 – Conteneurisation avec Docker

Application Multi-Conteneurs : Nginx · PostgreSQL · pgAdmin

Effectuée par :
MOUGUERT Meryem

Encadré par :
Pr. ELBAGHAZAOUI Bahaa Eddine

Introduction

Ce TP a pour objectif de dockeriser une application web multi-conteneurs composée de trois services :

- Un serveur Nginx pour servir le frontend (page HTML statique)
- Une base de données PostgreSQL pour le backend
- pgAdmin pour administrer la base de données via une interface web

Docker Compose est utilisé pour orchestrer ces trois conteneurs au sein d'un réseau partagé.

Étape 1 – Structure du projet

La structure de répertoires créée est la suivante :

```
project/
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   └── .dockerignore
├── backend/
│   └── db.env
└── docker-compose.yml
```

frontend/ : Contient le Dockerfile, la page HTML et le fichier .dockerignore

backend/ : Contient le fichier db.env avec les variables d'environnement PostgreSQL

docker-compose.yml : Orchestre les trois services : frontend, db et pgAdmin

Étape 2 – Fichiers du Frontend

index.html

Page HTML statique servie par Nginx :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dockerized App</title>
  </head>
  <body>
    <h1>Bienvenue dans l'application Dockerisée</h1>
  </body>
</html>
```

Dockerfile

Le Dockerfile crée une image basée sur Nginx Alpine (légère) et copie la page HTML :

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

FROM nginx:alpine : Image de base Nginx légère basée sur Alpine Linux

COPY : Copie index.html dans le répertoire servi par Nginx

EXPOSE 80 : Expose le port 80 du conteneur

CMD : Lance Nginx en mode foreground (nécessaire pour Docker)

FROM nginx:alpine : Image de base Nginx légère basée sur Alpine Linux

COPY : Copie index.html dans le répertoire servi par Nginx

EXPOSE 80 : Expose le port 80 du conteneur

CMD : Lance Nginx en mode foreground (nécessaire pour Docker)

.dockerignore

Exclut les fichiers inutiles lors de la construction de l'image :

```
*.log
```

Étape 3 – Configuration de la base de données

backend/db.env

Variables d'environnement pour PostgreSQL :

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp
```

POSTGRES_USER=Nom d'utilisateur de la base de données

POSTGRES_PASSWORD=Mot de passe de l'utilisateur

POSTGRES_DB=Nom de la base de données créée automatiquement

Étape 4 – Fichier docker-compose.yml

Le fichier docker-compose.yml orchestre les trois services :

```

services:
  frontend:
    build:
      context: ./frontend
    ports:
      - "8080:80"
    networks:
      - app-network

  db:
    image: postgres:13
    env_file:
      - ./backend/db.env
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network

  pgadmin:
    image: dpage/pgadmin4
    environment:
      - PGADMIN_DEFAULT_EMAIL=admin@admin.com
      - PGADMIN_DEFAULT_PASSWORD=admin
    ports:
      - "5050:80"
    depends_on:
      - db
    networks:
      - app-network

volumes:
  db-data:

networks:
  app-network:

```

Élément	Rôle
Frontend :	Construit depuis le Dockerfile local, accessible sur le port 8080
Db :	Image PostgreSQL 13 officielle, données persistées dans un volume
Pgadmin :	Interface graphique pour administrer PostgreSQL, port 5050
app-network :	Réseau Docker partagé permettant la communication entre conteneurs
db-data (volume) :	Volume persistant pour les données PostgreSQL — survivent aux redémarrages
depends_on :	pgAdmin attend que le conteneur db soit démarré avant de lancer

Étape 5 – Lancement et validation

Commande de lancement

```
docker-compose up --build
```

Cette commande effectue les opérations suivantes :

- Télécharge les images Docker nécessaires (postgres:13, dpage/pgadmin4)
- Construit l'image personnalisée du frontend depuis le Dockerfile
- Crée le réseau app-network et le volume db-data
- Lance les trois conteneurs simultanément

Résultat 1 – Frontend accessible sur http://localhost:8080

Bienvenue dans l'application Dockerisée

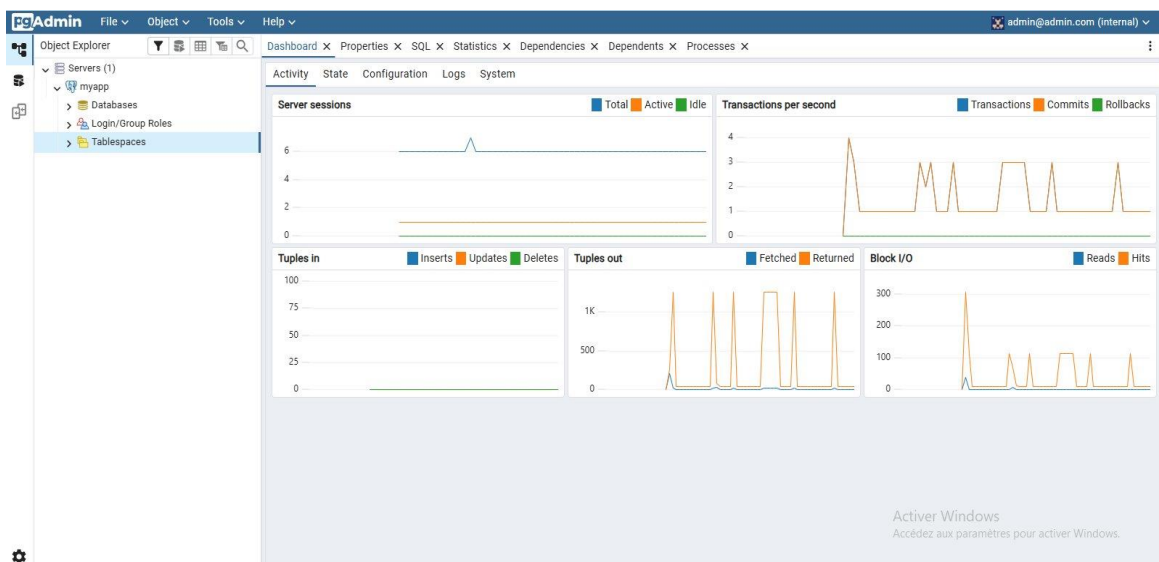
Activer Windows
Accédez aux paramètres pour activer Windows.

Résultat 2 – Conteneurs en cours d'exécution dans Docker Desktop

☐	☒	project	-	-	-	6.52%	5 hours ago			
☐	☒	frontend-1	7371e269268a	project-frontend		0%	5 hours ago			
☐	☒	db-1	62e5e6dc8159	postgres:13		0.02%	5 hours ago			
☐	☒	pgadmin-1	eea0c18afc92	dpage/pgadmin4	5050:80	6.5%	5 hours ago			

Figure 2 : Les 3 conteneurs (frontend-1, db-1, pgadmin-1) en état Running dans Docker Desktop

Résultat 3 – pgAdmin connecté à la base de données PostgreSQL



Connexion à pgAdmin

Paramètres de connexion utilisés dans pgAdmin :

URL pgAdmin : http://localhost:5050

Host (serveur DB) : db

Port : 5432

Database : myapp

Étape 6 – Volume persistant

Un volume Docker nommé db-data est configuré pour assurer la persistance des données PostgreSQL :

```
volumes:  
  db-data:/var/lib/postgresql/data
```

Avantages du volume persistant :

- Les données survivent à l'arrêt et au redémarrage des conteneurs
- Commande docker-compose down ne supprime pas les données (contrairement à docker-compose down -v)
- Les données sont stockées sur le système hôte et non dans le conteneur éphémère

Conclusion

Ce TP a permis de maîtriser les concepts fondamentaux de Docker :

Dockerfile : Création d'une image personnalisée Nginx pour servir le frontend

docker-compose : Orchestration de 3 services interdépendants en une seule commande

Réseaux Docker : Communication entre conteneurs via un réseau bridge partagé

Volumes Docker : Persistance des données PostgreSQL entre les redémarrages

Variables d'env : Configuration sécurisée via fichier .env séparé du code